instance variables that happen to belong to an object of class Class. Because of that, a Class Instance Variable can be accessed only by the class itself—not by an instance or by a subclass.

We've touched on many things: the current class, class definitions, self, class_eval, Class Instance Variables.... Now you can go back to Bookworm and put these features together.

Class Variables

If you want to store a variable in a class, you have more options than just using a *Class Instance Variable (109)*. You can also use a *class variable*, identified by an @@ prefix:

```
class C
  @@v = 1
end
```

Class variables are different from Class Instance Variables because they can be accessed by subclasses and by regular instance methods. (In that respect, they're more similar to Java's static fields.)

```
class D < C
  def my_method; @@v; end
end

D.new.my_method  # => 1
```

Unfortunately, class variables have a nasty habit of surprising you. Here's an example:

```
@@v = 1

class MyClass
  @@v = 2
end

@@v  # => 2
```

You get this result because class variables don't really belong to classes—they belong to class *hierarchies*. Because @@v is defined in the context of main, it belongs to main's class Object...and to all the descendants of Object. MyClass inherits from Object, so it ends up sharing the same class variable. As technically sound as this behavior is, it's still likely to trip you.

Because of unwelcome surprises like the one shown earlier, most Rubyists nowadays shun class variables in favor of Class Instance Variables. Also, Ruby 2.x issues a stern warning whenever you access a class variable from the top level.